Multi-Agent Computational Framework (Xogito) — Specification (Condensed)

July 6, 2026

1 Big picture

Multi-agent systems end up talking constantly about "beliefs": what the system thinks is true, what changed, and what counts as settled. The word hides three different things; if you mix them up you get either:

- **Freeze**: nothing updates because "beliefs" are treated as laws.
- **Drift**: laws and assumptions quietly change because they were treated as revisable beliefs.

The framework fixes this by separating three belief-shaped types.

1.1 A helpful analogy (courtroom)

Structural invariants are procedure (the judge does not rewrite them mid-trial). **Contextual anchors** are stipulated facts for *this* case (stable unless explicitly amended). **Epistemic claims** are the jury's evolving view (supposed to move as evidence arrives).

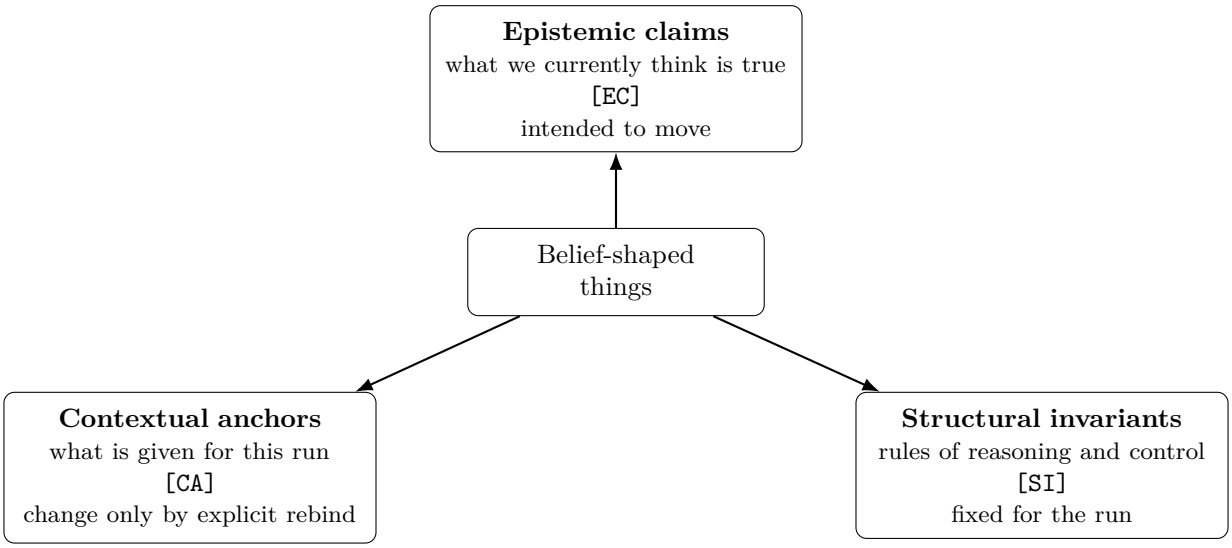

Figure 1: Three kinds of belief-shaped objects.

2 Core objects

These are the objects the rest of the spec keeps pointing back to.

- **Problem Specification** `[CA]`: goal, constraints, assumptions, success criteria.
- **Policy Base** `[CA]`: optional standing preferences (soft biases).
- **Task Graph**: dependency graph of tasks; *shape rules* are `[SI]`, current contents are bookkeeping.
- **Workspace**: container for artifacts and provenance; artifact *contents* are `[EC]`.
- **Belief Ladder** `[SI]`: `Refuted < Doubtful < Plausible < Supported < Corroborated < Established`, plus `Contested`.
- **Iteration Carry-Over Packet**: `[SI]` definition; per-iteration contents are a snapshot (mixed `[EC]` and `[CA]`).

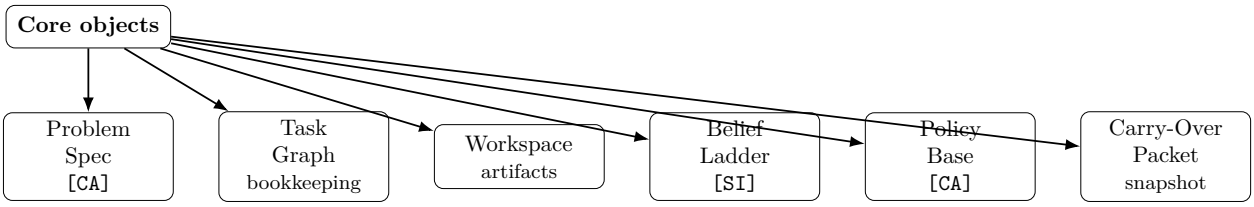

Figure 2: Core objects referenced throughout the framework.

3 System architecture

Everything described so far — claims, invariants, anchors, and the objects they attach to — has to run somewhere. This section describes the runtime shape: who does reasoning, who does bookkeeping, and how the two are kept separate.

3.1 A single Orchestrator (and why)

An **agent** is an independent reasoning context: it investigates, weighs evidence, and produces artifacts. The **Orchestrator** is not an agent. It does not reason about the user's domain problem; it reasons about the *computation* that is trying to solve the user's problem.

New reasoning contexts are introduced only when they earn their cost:

- **Context isolation:** a task must be insulated from another task's assumptions/framing.
- **Independent reasoning:** a claim must be checked by a context that did not produce it (critic/adjudication).

3.2 Architecture diagram

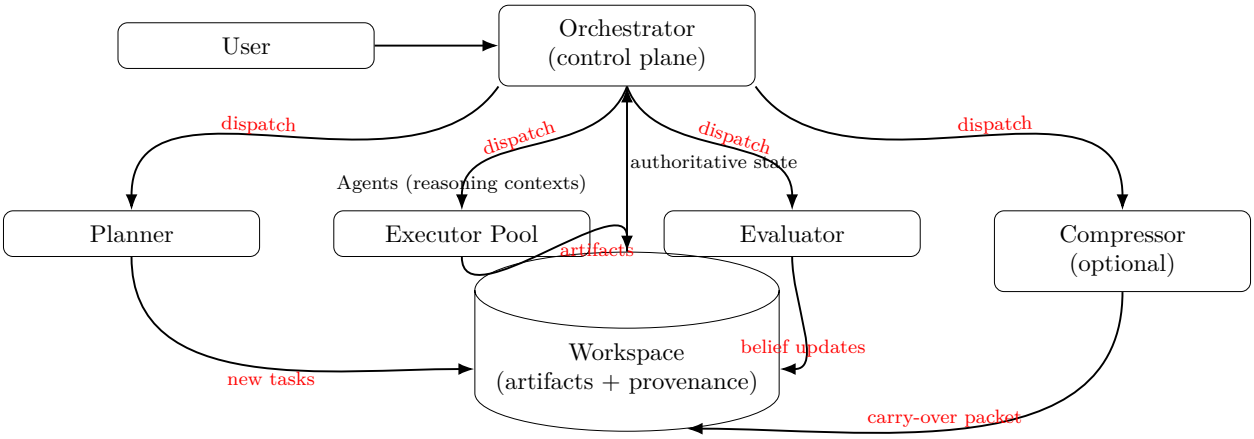

Figure 3: Runtime architecture: one Orchestrator controlling a small set of reasoning components; persistent state lives in the Workspace.

3.3 Communication model (thin on purpose)

- Agents do **not** message each other directly.
- Agents do **not** own shared persistent state.
- The Workspace is the shared artifact repository; the Orchestrator owns the workflow.
- Agent I/O is strictly: task from Orchestrator → output → Orchestrator → validated artifact in Workspace.

3.4 Component responsibilities

- **Orchestrator:** maintains Task Graph + Workspace, schedules eligible tasks, dispatches work, validates outputs against invariants, runs the iteration/checkpoint loop, produces the final report.
- **Planner:** decomposes goals/questions into new task objects with dependencies.
- **Executor Pool:** executes ready tasks (investigation/computation/synthesis) and emits a single artifact with evidence.
- **Evaluator:** applies contradiction detection, weakest-link propagation, evidence-combination rules, and success-criteria checks.
- **Compressor (optional):** compacts the iteration into the Carry-Over Packet (what must persist to continue correctly).
- **Workspace:** stores validated artifacts + provenance; supports multiple belief states for contested claims.

3.5 Orchestrator determinism (mostly)

Most orchestration steps are mechanical given the current state: maintaining graphs, enforcing invariants, dispatching eligible tasks, counting iterations, checking stop conditions, and assembling the final report. A small number of decisions are meta-judgments (e.g., prioritizing ready tasks, deciding whether input is formalizable), but they are still judgments about *running the computation*, not judgments about the user's domain answer.

4 Epistemic substrate (how claims move)

4.1 Bad example: fake precision (and the fix)

Bad example: an agent writes "confidence = 0.78" and the system later averages several such numbers. These decimals are not grounded in repeated trials or a calibrated distribution; they're vibes wearing a decimal point.

Fix: every claim uses a categorical **Belief Ladder** (`Refuted < Doubtful < Plausible < Supported < Corroborated < Established`) plus `Contested`. Movement is discrete and event-based.

4.2 Artifact shape (fixed)

Artifacts use a fixed schema; fields like `claim`, `belief_state`, `evidence`, `numeric_estimate`, `inherited_uncertainty` are epistemic content, while IDs and provenance are bookkeeping.

4.3 Bad example: scoring evidence (and the fix)

Bad example: compute something like `evidence_strength = weight` $\times$ `reliability`. This assumes a real number line the system never honestly produces.

Fix: describe each evidence item using categorical attributes.

4.4 Evidence attributes (categorical)

Attribute	Values (examples)	Notes
Reliability	untrusted, unverified, verified_once, verified_independently	judgment (`[EC]`)
Directness	direct_observation, direct_computation, inference, speculative	judgment (`[EC]`)
Independence	independent, overlapping, redundant	computed from provenance
Relevance	direct, partial, tangential	judgment (`[EC]`)
Novelty	new, restated	judgment (`[EC]`)

4.5 Promotion, demotion, and conflict

- Promotion/demotion is discrete: a qualifying independent evidence event moves a claim by *one rung* (never more).
- Redundant evidence never counts twice (computed by provenance).
- If a high-confidence claim (`Corroborated/Established`) is directly contradicted by a strong independent observation/computation, escalate to adjudication (do not quietly average or demote).
- If adjudication cannot decide, preserve the disagreement as `Contested` and propagate that flag downstream.

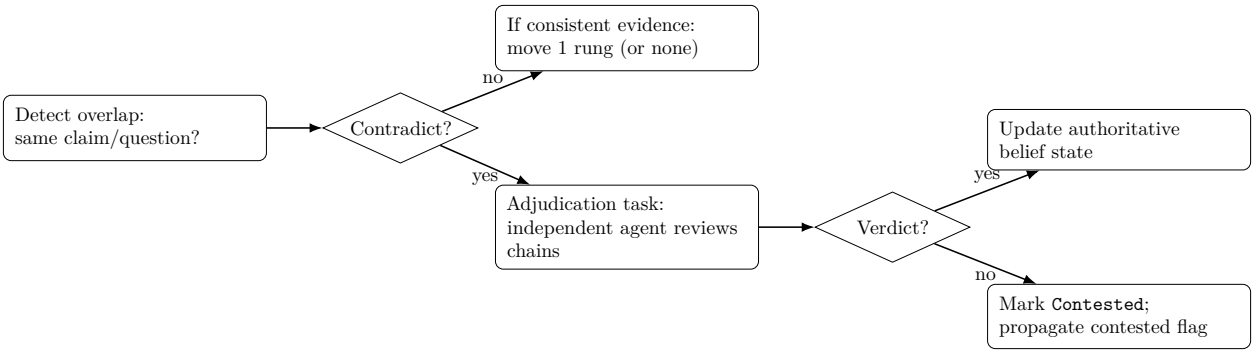

Figure 4: Conflict handling at a glance.

4.6 Propagation

Derived artifacts obey a weakest-link cap: a child claim cannot exceed the minimum belief state of its critical parents. Any unresolved ancestor uncertainty is carried explicitly (no numeric decay).

5 Execution loop (phases)

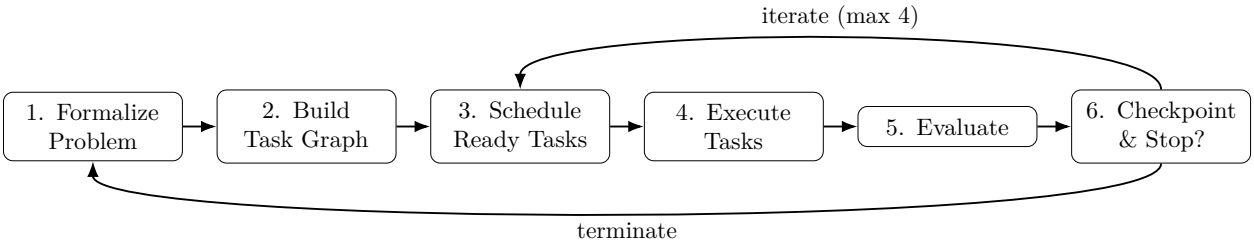

Figure 5: Phase order and iteration loop.

6 Agent interaction (end-to-end)

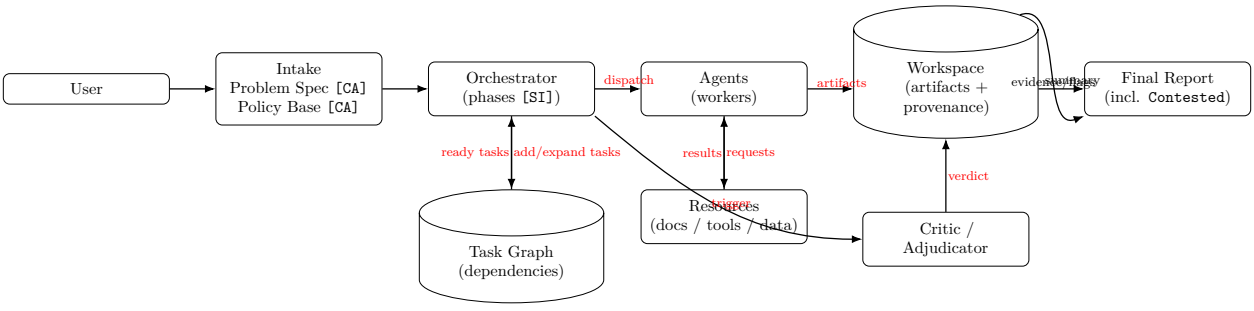

Figure 6: End-to-end interaction: intake anchors, orchestrated tasks, agent work, enforced critique/adjudication, and reporting.

7 End-to-end workflow (single-page diagram)

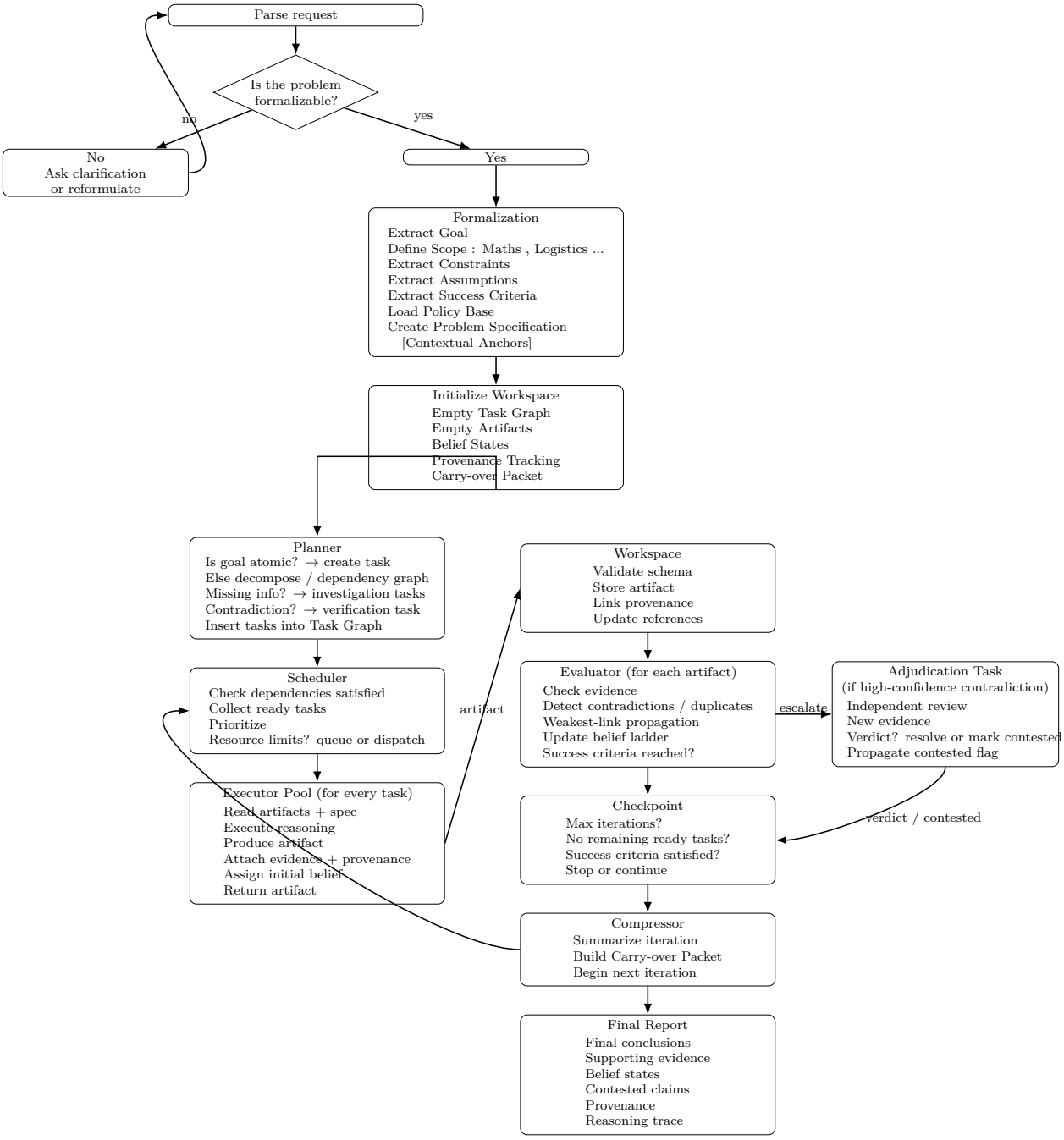

Figure 7: Single-page workflow diagram matching the provided text flow.